Machine Learning Course by IBM

What is Machine Learning?
It is a sub-field of computer science that allows computers to learn without being explicitly programmed.

Major Machine Learning Techniques:
- **Regression / Estimation:** Predicting continuous values

- **Classification:** Predicting the class/category of the case

- **Clustering:** Finding the structure of data; summarisation

- **Associations:** Associating frequent co-occurring items/events

- **Anomaly Detection:** Discovering abnormal or unusual cases

- **Sequence mining:** Predicting next events; click-stream (Markov Model, HMM)

- **Dimension Reduction:** Reducing the size of data (PCA)

- **Recommendation Systems:** Recommending items

The difference between Artificial Intelligence, Machine Learning and Deep Learning:
Artificial Intelligence tries to make computers intelligent to mimic the cognitive functions of humans. Examples: Computer Vision, Language Processing, Creativity etc.

Machine Learning covers the statistical part of **AI**. It teaches the computer to solve the problem by looking at hundreds of problems. Examples: Classification, Clustering, Neural Networks etc.

Deep Learning is a very special field where computers can make their own decisions. It is a revolution!

Python Libraries for Machine Learning:

- **NumPy**
- **SciPy**
- **Matplotlib**
- **PanDas**
- **SciKit Learn:** It works with NumPy and SciPy and also includes Machine Learning algorithms like Classification, Regression and Clustering. It is easy to implement!

Supervised vs Unsupervised Machine Learning

Supervised Machine Learning occurs when we observe and direct the execution of a task, project or activity. We do this by teaching the machine so that it can predict the unknown data for future instances. We teach it by training it on a dataset. Examples of **Supervised Machine Learning** are **Classification** and **Regression**.

Unsupervised Machine Learning occurs when we let the model work on its own to discover information that may not be visible to humans. Generally, they are more difficult algorithms than **Supervised Machine Learning**. Examples of **Unsupervised Machine Learning** are **Dimension Reduction**, **Density estimation**, **Market Basket Analysis** and **Clustering**.

Regression

It is the process of predicting continuous values. In Regression, there are 2 types of variables i.e. dependent variable and one or more independent variables. The **Dependent variable (Y)** can be seen as the state, target or final goal. The **independent variable (X)** can be seen as the cause of those states.

In **Regression**, the **Dependent variable (Y)** should be a continuous value and cannot be a discrete value.

There are 2 types of Regression Model:

- Simple Regression: When one independent variable estimates a dependent variable. It can either be linear or non-linear

- Multiple Regression: When more than 1 independent variable is present.

Applications of Regression:

- Sales Forecasting
- Satisfaction Analysis
- Price Estimation
- Employment Income

Simple Linear Regression

Here, we are given only 1 **Independent variable** used to estimate a **Dependent variable**. Let's take the example of predicting the **CO2 Emission** of cars according to their **Engine size**. So, the **Independent variable Engine Size** is represented by **X1** and the **Dependent variable CO2 Emission** is denoted by **Y1.**

Now, we will plot a best-fit continuous line on the graph using the previous data points which is most suitable according to the scatter plot of the earlier data points. The form of the model would be

$$\hat{y} = \Theta_0 + \Theta_1 X_1$$

$\hat{y}$ = Response Variable

Θ_1 = Slope of the fitting line
Θ_0 = Intercept
X_1 = Single Predictor

How to find the Best Fit?
The objective of **Linear Regression** is to minimise the MSE (Mean Squared Error) and to minimise it, we should find the best parameters Θ_0 and Θ_1.

$$\theta_1 = \frac{\sum_{i=1}^{s}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{s}(x_i - \bar{x})^2}$$

$\bar{x}$ = Average value of the engine size
$\bar{y}$ = Average value of the CO2 Emission

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

Now, we can plot the best-fit line for our model. This equation can also be used to predict the CO2 Emission for a given value of Engine Size.

Advantages of Linear Regression
- Very fast
- No parameter tuning
- Easy to understand, and highly interpretable

<u>Model Evaluation</u>
- Train and Test on the same Dataset: Here we train a model on the entire dataset and then we test it on a particular portion of the dataset. This evaluation approach has higher accuracy.

- Train/Test Split: Here, we select a portion of data and test it on the other portion of the dataset. It has a more accurate evaluation of out-of-sample accuracy. The testing dataset is not part of the training data.

What is training and out-of-sample accuracy?

Training Accuracy: High accuracy is not necessarily a good thing. Result of overfitting; the model is overly trained to the dataset, which may capture noise and produce a non-generalised model.

Out-of-sample Accuracy: Our model must have a high, out-of-sample accuracy. We can improve it by using another evaluation approach called **Train/Test Split**.

How to use K-fold cross-validation?

Here, the model is trained on a particular part of the dataset and tested on the remaining dataset. It is done 'k' times and each time accuracy of the model is compared. The resulting accuracy is the average of the accuracies acquired after each fold.

What is an error in the model?

It is the difference between the data point and the trend line generated by the algorithm.

- Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j|$$

- Mean Squared Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Root Mean Square Error

$$\mathrm{R}MSE = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2}$$

- **Relative Absolute Error**

$$RAE = \frac{\sum_{j=1}^{n} |y_j - \hat{y}_j|}{\sum_{j=1}^{n} |y_j - \bar{y}|}$$

- **Relative Squared Error**

$$RSE = \frac{\sum_{j=1}^{n} (y_j - \hat{y}_j)^2}{\sum_{j=1}^{n} (y_j - \bar{y})^2}$$

Multiple Linear Regression

In reality, multiple factors predict a particular value. For example, CO_2 Emission is affected by both the Engine Size and the Fuel Consumption of the vehicle.

Examples of Multiple Linear Regression:

- **Independent variable's effectiveness on prediction**: Does revision time, test anxiety, lecture attendance and gender have any effect on the exam performance of students?

- **Predicting Impacts of Changes**: How much blood pressure goes up (or down) for every unit increase (or decrease) in the BMI of a patient?

Multiple Linear Regression is very useful because you can examine which variables are significant predictors of the outcome variable. Also, you can find which feature impacts the outcome variable.

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ ... + \theta_n x_n$$

$$\hat{y} = \theta^T X$$

Remember: $\boldsymbol{\Theta^T X}$ in 1D is the equation of the line

How to find Optimized Parameters?

Optimized Parameters are the ones which lead to a model with fewer errors. Here, we predict the value for a particular row and find the error between the actual and the predicted value.

Using MSE to expose the errors in the model

Although it is not the most accurate method to minimise the method, a very popular method to minimise the error.

$$\hat{y} = \theta^T X$$

$\hat{y}_i = 140$ the predicted emission of x_i

$y_i = 196$ actual value of x_i

$y_i - \hat{y}_i = 196 - 140 = 56$ residual error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Estimate Multiple Linear Regression Parameters

How to estimate Θ?

- **Ordinary Least Squares**
    - Linear Algebra operations
    - Takes a long time for large datasets (10K+ rows)
- **An Optimization Algorithm**
    - Gradient Descent
    - Proper approach if you have a very large dataset

After we found the parameters of the linear equation, making predictions is as simple as solving the equation for a specific set of inputs.

How to determine whether to use Simple Linear Regression or Multiple Linear Regression?

Sometimes, **Simple Linear Regression** is considered better than **Multiple Linear Regression** if there is only one independent variable to predict the dependent variable.

How many independent variables should you use?
Adding too many independent variables without any justification may result in an overfit model.

Should the independent variable be continuous?
Categorical independent variables can be incorporated into a regression model by converting them into numerical variables.

Remember: There needs to be a linear relationship between the dependent variable and each of your independent variables. We can scatter plots and then visually check for linearity.

Classification

It is a Supervised Machine Learning approach which is a means of categorizing some unknown items into a discrete set of categories or classes. It attempts to learn the relationship between a set of feature variables and a target variable of interest. The target attribute is a categorical variable with discrete values.

How does Classification work?

It determines the class label for an unlabeled test case. There are 2 types of classifications i.e. **Binary Classification** and **Multi-class Classification**.

Binary Classification: When we want to predict whether a person can pay the debt in banks, we use certain other factors like age, education, employment, income, previous debts, etc. Then, we can predict true or false (0 and 1)

Multi-class Classification: When we want to predict the type of drug for the patient among the 3 drugs, we will use **Multi-class Classification**. This will help to predict multiple drugs according to the features available.

Classification use cases

Which category does a customer belong to?
Whether a customer switches to another provider/brand?
Whether a customer respond to a particular advertising campaign?

Classification Applications

- Email Filtering
- Speech Recognition
- Handwriting Recognition
- Biometric Identification
- Document Classification

Classification Algorithms in Machine Learning
- Decision Trees (ID3, C4.5, C5.0)
- Naive Bayes
- Linear Discriminant Analysis
- k - nearest Neighbours
- Logistic Regression
- Neural Networks
- Support Vector Machines (SVM)

K - Nearest Neighbours

If a Telecom Company provides the 4 services among the customers, then it is a classification problem because we have to determine on what basis the categorical services are given to the customers and we have to determine the service type for the new customers as well using the previous dataset.
Here we will use a specific type of classification i.e. k - Nearest Neighbour

Determining the class using 1st KNN
We will use fields as predictors and then plot customers based on their group membership. Now, we have a new customer with known fields, How can we find the class of the customer?
We can find the closest neighbour of the new customer and say that he/she probably belongs to this group because its nearest neighbour belongs to that group as well. This approach needs to be corrected.

Let's take the 5 nearest neighbours and do a majority vote among them to define the class of our new customer. We will assign the majority of cases to the new customers as it makes more sense!

What is k - Nearest Neighbour (or KNN)?
It is a classification Algorithm that takes a bunch of label points and uses them to learn how to label other points. It classifies cases based on their similarities to other cases. Cases that are near each other are said to be neighbours. Based on **similar cases with the same class labels are near each other**.
There are different ways to calculate the similarity or conversely the distance or dissimilarity of 2 data points. This can be done using **Euclidean Distance**.

How does this algorithm work?
- Pick a value for K
- Calculate the distance of the unknown case from all cases
- Search for the k-observations in the training data that are "nearest" to the unknown data point
- Predict the response of the unknown data point using the most popular response value from the k-nearest neighbours

Calculating the similarity/distance in a 1D space
We will use **Minkowski Distance** to calculate the distance

$$\text{Dis}\ (x_1, x_2) = \sqrt{\sum_{i=0}^{n}(x_{1i} - x_{2i})^2}$$

It is **Euclidean Distance**.

Calculating the similarity/distance in a 2D space
We will use the same formula but, in 2D space

$$\text{Dis}\ (x_1, x_2) = \sqrt{\sum_{i=0}^{n}(x_{1i} - x_{2i})^2}$$

What is the best value of K in KNN?
If we take K=1, it will be a bad decision because it might catch the noise from the dataset or we can say that got an anomaly. A low value of K causes a highly complex model as well which might

result in overfitting of the model. It means the predicting process needs to be generalized more to be used for out-of-sample cases. **Out-of-sample** data is data that is outside of the data set used to train the model.
Remember: Overfitting is bad as we want the model that works on any data.
If we take a very high value for K, then the model will become highly generalized.

So, how to get the best value of K?
The general solution is to reserve a part of your data for testing the accuracy of the model. Once you've done so, choose K equals one and then use the training part for modelling and calculate the accuracy of prediction using all samples in your test set. Repeat this process increasing the K and see which K is best for your model.

Computing continuous targets using KNN
KNN can also be used for Regression. For example: While predicting the price of the house, we can use KNN to find the nearest house and the nearest house based on the features that help us predict the price using the similarities.

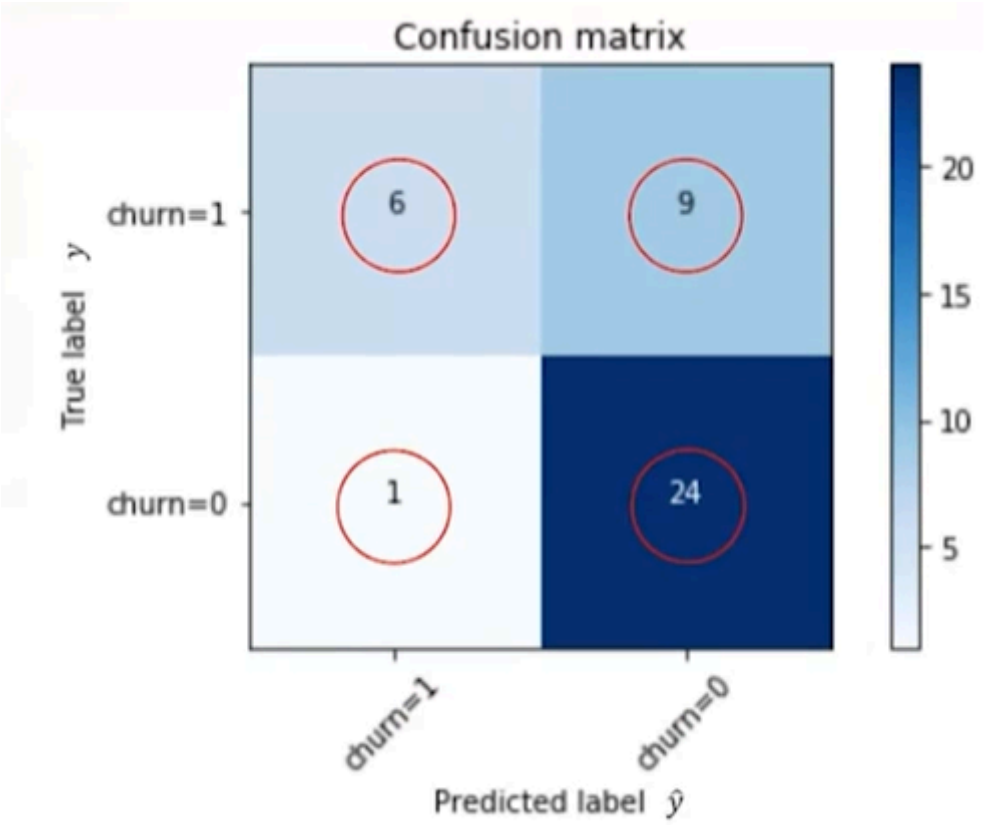
Evaluation Metrics in Classification
Here, we will calculate the accuracy of the model by comparing the predicted values with the actual values.

- **Jaccard Index**
ŷ: Predicted labels
y: Actual labels

$$J(y, \hat{y}) = \frac{|y \cap \hat{y}|}{|y \cup \hat{y}|} = \frac{|y \cap \hat{y}|}{|y| + |\hat{y}| - |y \cap \hat{y}|}$$

- **F1 - Score**

Here, we will use the confusion matrix for calculating the accuracy of the model. It is ideal for the model.



Remember the concepts of True-Positive (TP), False-Negative (FN), False-Positive (FP) and True-Negative (TN).

- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- F1-score = 2x (prc x rec) / (prc+rec)

Remember: Jaccard Index and **F1-Score** can be used for **Multi-Class Classifiers** as well.

- **Log Loss**

Performance of a classifier where the predicted output is a probability value between 0 and 1.

$$LogLoss = -\frac{1}{n}\sum(y \times \log(\hat{y}) + (1-y) \times \log(1-\hat{y}))$$

Decision Trees

Imagine being a person who is looking after patients with the same illness and you have to prescribe the drug to them. You are given a dataset containing factors like age, sex, BP, and cholesterol. Using that, build a **Decision Tree** model.

It is built by splitting the training set into distinct nodes, where one node contains all or most of one category of the data. **Decision Trees** are about testing an attribute and branching the cases based on the result of the test. Each internal node corresponds to a test, and each branch corresponds to the result of the test, and each leaf node assigns a patient to a class.

Decision Tree Learning Algorithm

It can be built by considering the attributes one by one. First, choose an attribute from our dataset and then, calculate the significance of the attribute in the splitting of data. Next, split the data based on the value of the best attribute, then go to each branch and repeat it for the rest of the attributes. After building this tree, we can use it to predict the class of unknown cases.

How do you build Decision Trees?

They are built using recursive partitioning to classify the data. The algorithm chooses the most predictive feature to split the data on. For example: in the case of drugs, the factor of gender/sex is more predictable as compared to the other features. After that, we have to make one more decision in the branch until the results get pure/clear.

The Impurity of nodes is calculated by the Entropy of data in the node. **What is Entropy?**

It is a measure of randomness or uncertainty. It depends on how much random data is in that node and is calculated for each node. The lower the Entropy, the less uniform the distribution, and the purer the nodes.

$$\text{Entropy} = - p(A)\log_2(p(A)) - p(B)\log_2(p(B))$$

While selecting the feature for decision-making, we have to measure the Entropy of that node and make sure the Entropy is less so that the node is pure.

Which attribute is the best?
We will choose the feature that has the highest information gain.
What is Information Gain? It is the information that can increase the level of certainty after splitting.

Information Gain = (Entropy before split) – (weighted entropy after split)

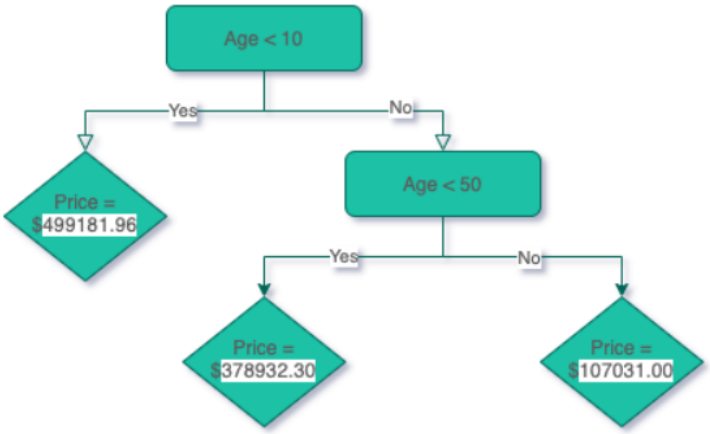
Gain (s, Sex)
= 0.940 – [(7/14)0.985 + (7/14)0.592]
=0.151

Regression Trees (Optional)

The basic idea behind Regression Trees is to split data into groups based on features, like in classification, and return a prediction that is the average across the data we have already seen.

Consider using age to predict the price of the house. Ages between 0 to 10 have an average price of $500,000, ages between 10 to 50 have an average price of $380,000, and ages over 50 years have an average price of $100,000.

Using the above data, we can create the **Regression Tree**. The prices are determined by calculating the average cost of the houses in the age range.



The trees are built similarly to classification, but instead of using the entropy criterion. In Classification Trees, we choose features that increase the information gain. In Regression Trees, we choose features that minimize the error.

A popular one is Mean Absolute Error,

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

How are Regression Trees built?

Take the dataset sample shown below, the first step is to decide what the first decision is. We will do this by using the criterion and checking every single feature in the dataset to see which one produces the minimal error.

Categorical Features

Categorical features are simple, here we have Near Water so all we need to do is calculate the error if we use this as the first feature. Near Water feature has two categories: ‘Yes’ and ‘No’, therefore, we must calculate the average ‘Price’ of houses in the ‘Yes’ and ‘No’ categories. Then we use those values to calculate the average error.

‘No’ Category:

Index of Houses in ‘No’ Category = [0, 1, 2, 3, 4]
Prices of Houses in ‘No’ Category = [260831.34, 222939.35, 101882.1, 226868.52, 94868.94]
Average House Price in ‘No’ Category = 181478.05
Absolute Error = [79353.29, 41461.30, 79595.95, 45390.47, 86609.11]

‘Yes’ Category:

Index of Houses in ‘Yes’ Category = [5,6,7,8,9]
Prices of Houses in ‘Yes’ Category = [197703.55, 347982.98, 343150.38, 206713.16, 329768.77]
Average House Price in ‘Yes’ Category = 285063.77
Absolute Error = [87360.22, 62919.22, 58086.61, 78350.61, 44705.00]

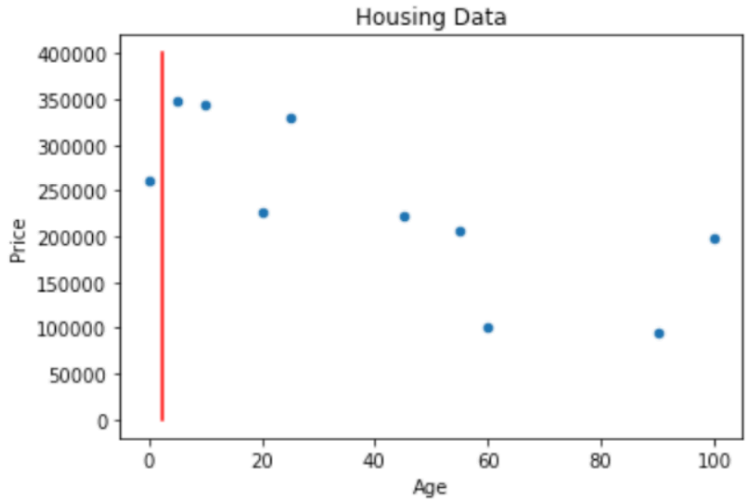
MAE

This is the MAE of the Near Water feature, and this number will be used to compare against the MAE of all other features to determine which one is the lowest. Therefore, it will establish the first decision in our regression tree.
MAE = 66383.17

Numerical Features

Numerical features, like ‘Age’, are trickier to handle because we need to find a number, instead of using a category, to split the data by. We do this by creating a boundary between each point, then we calculate the error.

For example, first, we create the boundary between the first two data points, which are (0, 260831.34) and (5, 347982.98), so we create a boundary of x = 2.5 (The midpoint between the x component of the first two data points). We now find the average price of the houses on the left and right sides of this boundary and use it to calculate the MAE.



Left:

Index of Houses on the Left Side = [0]
Prices of Houses on the Left Side = [260831.34] Average House Price of Left Side = 260831.34 Absolute Error = [0]

Right:

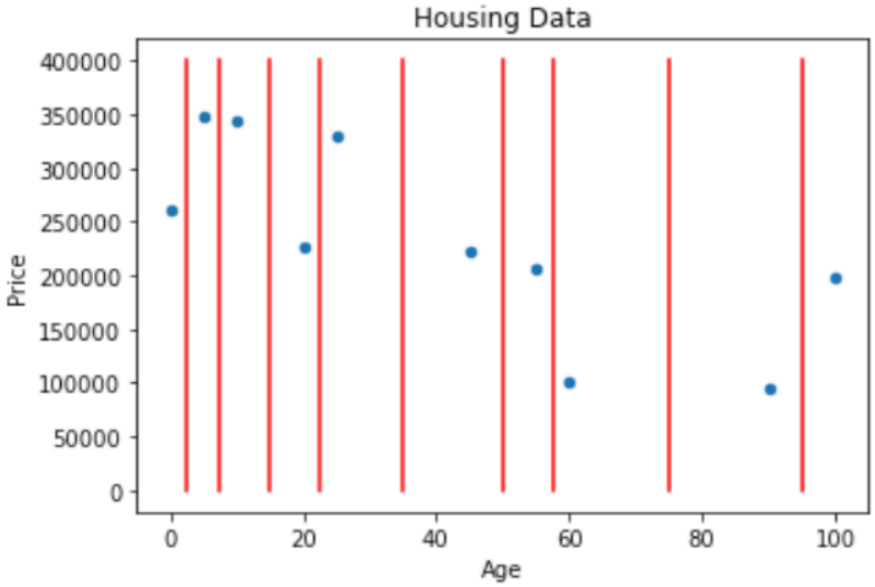
Index of Houses on the Right Side = [1, 2, 3, 4, 5, 6, 7, 8, 9]
Prices of Houses in the Right Category = [222939.35, 101882.1, 226868.52, 94868.94, 197703.55, 347982.98, 343150.38, 206713.16, 329768.77]
Average House Price in the Right Category = 230208.64
Absolute Error = [7269.29, 128326.54, 3340.12, 135339.7, 32505.09, 117774.34, 112941.74, 23495.48, 99560.13]

MAE

Now we can find the MAE, using the absolute errors from the left and right sides.
MAE = 66055.24

Further Steps

This process is then repeated for each boundary between each pair of consecutive points.



Choosing the Decision

Now, we compare the categorical MAE and the lowest numerical MAE, in this case, the categorical is 66055.24, and the numerical is 49726.55. So, for the first decision, we will use the numerical 'Age' feature.

When do we Stop?

With the regression tree above, we have two options, we can either stop here and use the average value of the 'Yes' (left) and 'No' (right) to predict the house prices, or we can continue to add more decisions to either branch. There are a few conditions that are commonly used to stop growing regression trees:

• Tree depth
• Number of remaining samples on a branch
• Number of samples on each branch if another decision is made

The depth of the tree above is 1 because there is a single decision and the number of samples on each side is 5. Let's add more decisions until the depth of the tree is 2. First, we start with the 'Yes'

(left) side and we calculate the MAE for the features using the houses that have ‘Age’ < 35.

Adding Decisions

Left

Like before, we use the Near Water feature and calculate the MAE on houses with indexes 0, 3, 6, 7, and 9.

Categorical Features

MAE = 11005.34

Numerical Features

Now, we find the MAE for the boundaries in the ‘Age’ feature.

Right

Now we will find the features that result in the lowest MAE from the houses with index 1, 8, 2, 4, and 5.

Categorical Features

MAE = 35018.94

Here, we can also see that the ‘Age’ feature will result in the lowest MAE with the boundary set to 57.5, so on the ‘No’ (right) side of the tree, we will add another decision for the ‘Age’ feature.

Final Result

Logistic Regression

It is a classification algorithm for categorical variables. If a company wants to predict which customer might leave us next month.

In **Logistic Regression**, we use one or more independent variables. It is analogous to **Linear Regression** but tries to predict the categorical or discrete target field instead of a numeric one. In this Regression, we predict the binary variable.

Here, the Independent variable should be continuous, if categorical they should be dummy or indicator coded meaning we have to transform them into some continuous value. It can also used for multi-class classification.

Applications of Logistic Regression

- Predicting the probability of a person having a heart attack
- Predicting the mortality in injured patients
- Predicting the failure of a product

When is Logistic Regression suitable?

- If your data is Binary
- If you need probabilistic results
- When you need a linear decision boundary
- If you need to understand the impact of a feature

Remember: We can also achieve a complex decision boundary using polynomial processing as well.

Logistic Regression vs Linear Regression

Using **Linear Regression**, we can predict the income of a person by using certain features but, categorical data is different from the numerical one. Although we can solve this problem of classification using **Linear Regression**, it is not accurate and we cannot get the probability.

When using this algorithm, there is a very sharp threshold line which classifies the value. Instead of having a sharp line, we would like to have a smooth line which makes it easy to classify.

If we take the Sigmoid of $\boldsymbol{\Theta^T X}$ in **Linear Regression** i.e.

$$\hat{y} = \sigma(\theta^T X)$$

This will make the Threshold Line smoother.

Sigmoid Function in Logistic Regression

It is also called the **Logistic Function**.

$$\sigma(\theta^T X) = \frac{1}{1 + e^{-\theta^T X}} \uparrow\downarrow$$

$\sigma(\theta^T X) = 1$

$\sigma(\theta^T X) = 0$

What is the output of our model?

$$P(Y=1|X)$$
$$P(y=0|X) = 1 - P(y=1|x)$$

The Training Process

$$\boldsymbol{\sigma(\theta^T X)} \longrightarrow \mathbf{P(y=1|x)}$$

1. Initialize Θ
2. Calculate $\hat{y} = \sigma(\Theta^T X)$ for a customer
3. Compare the output of ŷ with the actual output of customer, y, and record it as an error
4. Calculate the error for all customers
5. Change the Θ to reduce the cost
6. Go back to step 2

Logistic Regression Training:

The main objective of **Logistic Regression** is to change the parameters of the model, to be the best estimation of the labels of the samples in the dataset.

General Cost Function

It is the difference between the actual values of **y** and our model output **ŷ**. This is a general rule for most cost functions in **Machine Learning**.

$$Cost(\hat{y}, y) = \quad \sigma(\theta^T X) - y$$

$$J(\theta) \ = \frac{1}{m}\sum_{i-1}^{m} Cost(\hat{y}, y)$$

How to find the best weights or parameters that minimize the cost function? We should calculate the minimum point of this cost function and it will show us the best parameters for our model.

Plotting the cost function of the model

- Model ŷ
- Actual value y=1 or 0
- If y=1 and ŷ=1, cost=0
- If y=1 and ŷ=0, cost = large

Since the **Cost Function** is difficult, we can also use the **Minus Log Function**.

$$Cost(\hat{y}, y) = \begin{cases} -\log(\hat{y}) & if\ y = 1 \\ -\log(1 - \hat{y}) & if\ y = 0 \end{cases}$$

Then, the **Total Cost Function** is

$$J(\theta) \ = -\frac{1}{m}\sum_{i-1}^{m} y^i \log(\hat{y}^i) + (1 - y^i)\log(1 - \hat{y}^i)$$

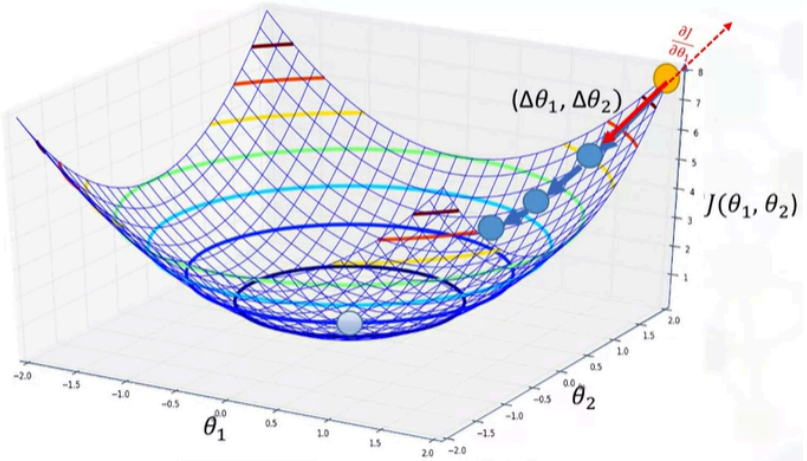
How to find the best parameters for our model?

By finding and minimizing the Cost Function of our model. In other words, to minimize the **J(Θ)** we just defined.

How to minimize the Cost Function?
By using an Optimization approach like **Gradient Descent**.

What is Gradient Descent?
It is an iterative approach to finding the minimum of a function. A technique to use the derivative of the **Cost Function** to change the parameter values, to minimize the cost.

Using Gradient Descent to minimize the cost
During the process, an Error Bowl is created by the **Total Cost Function**. We have to find the point which has the least error in that bowl by changing the parameters. To decrease **J**, we should move in the opposite direction. This means to move in the direction of the negative derivative for $\mathbf{\Theta_1}$ i.e. Slope.



If the slope is large, we should take a large step because we are far from the minimum. If the slope is small, we should take a smaller step. **Gradient Descent** takes increasingly smaller steps towards the minimum with each iteration

$$\frac{\partial J}{\partial \theta_1} = -\frac{1}{m}\sum_{i=1}^{m}(y^i - \hat{y}^i)\,x_1^i$$

Training Algorithm Recap
1. Initialize the parameters randomly

$$\theta^T = [\theta_0, \theta_1, \theta_2, \ldots.]$$

2. Feed the Cost Function with the training set, and calculate the error

$$J(\theta) = -\frac{1}{m}\sum_{i-1}^{m} y^i \log(\hat{y}^i) + (1 - y^i)\log(1 - \hat{y}^i)$$

3. Calculate the Gradient of Cost Function

$$\nabla J = [\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \frac{\partial J}{\partial \theta_3}, \ldots, \frac{\partial J}{\partial \theta_k}]$$

4. Update weights with new values

$$\theta_{new} = \theta_{prv} - \eta \nabla J$$

5. Go back to step 2

6. Predict the new customer X

$$\mathbf{P(y=1|x)} = \sigma(\boldsymbol{\theta}^T \boldsymbol{X})$$

Support Vector Machine

It is a classification algorithm. It can be used to predict Cancer in patients by using the previous data i.e. understanding the pattern within the data that might show, benign or malignant cells.
A **Support Vector Machine** is a supervised algorithm that can classify cases by finding a separator.
- Mapping data to a High-dimensional feature space
- Finding a separator

The **SVM** algorithm outputs an optimal hyperplane that categorizes new examples.

Data Transformation

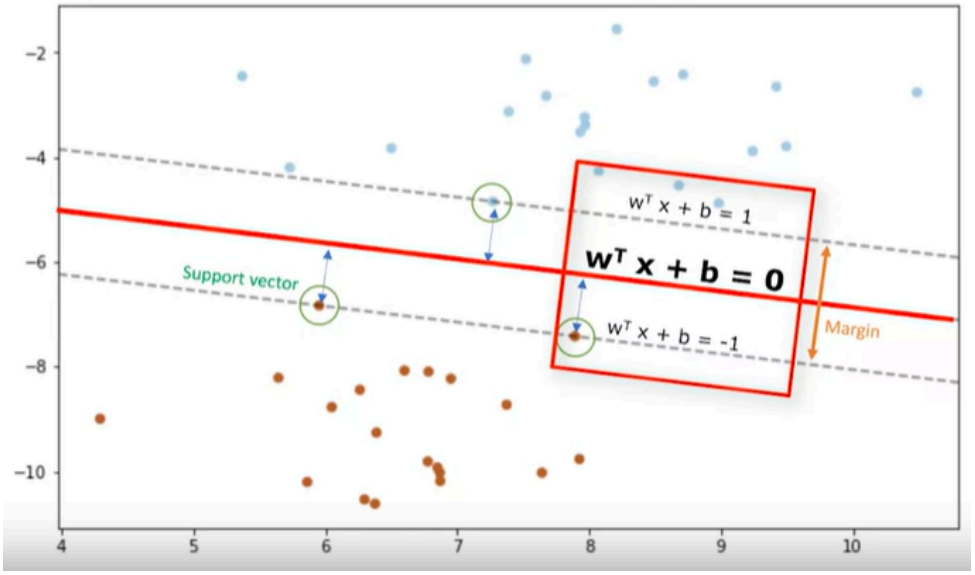
For simplicity, we will take only 1 independent feature. Since it is not linearly separable, so will transfer into a 2D space,
Kernel Function,

$$\phi(x) = [x, x^2]$$

Now, data will be linearly separable. The **Hyperplane** is a line dividing a plane into 2 parts where each class lays on either side.
Now, we can use this line to classify new cases.
The **Kernelling** can be **Linear**, **Polynomial**, **Radial Basis Function**, and **Sigmoid**. We usually use different functions and compare the results.
Remember: **Kernelling** means transforming data into a higher dimension.

Using SVM to find the Hyperplane

The goal is to choose a Hyperplane with as big a margin as possible. This can be done using the Support Vectors.



Advantages of SVM:
- Accurate in high-dimensional spaces
- Memory efficient

Disadvantages of SVM:
- Prone to over-fitting
- No probability estimation
- Small datasets

Applications of SVM:
- Image Recognition
- Text category assignment
- Detecting spam
- Sentiment Analysis
- Gene Expression Classification

Multiclass Prediction

SoftMax Regression

One-vs-All & One-vs-One for Multi-class Classification

In Multi-class classification, we classify data into multiple class labels. Unlike classification trees and nearest neighbours, the concept of Multi-class classification for linear classifiers is not as straightforward. We can convert logistic regression to Multi-class classification using multinomial logistic regression or SoftMax regression; this is a generalization of logistic regression. SoftMax regression will not work for Support Vector Machines (SVM); One vs. All (One-vs-Rest) and One vs One are two other multi-class classification techniques that can convert most two-class classifiers to a multi-class classifier.

SoftMax Regression

It is similar to **Logistic Regression**, the **SoftMax** function converts the actual distances i.e. dot products of ***x*** with each parameter $\boldsymbol{\Theta_i}$ for ***k*** classes in the range from 0 to ***k***-1. This is converted to probabilities using the following formula.

$$\mathrm{softmax}(x, i) = \frac{e^{-\theta_i^T x}}{\sum_{j=1}^{K} e^{-\theta_j^T x}} \quad (1)$$

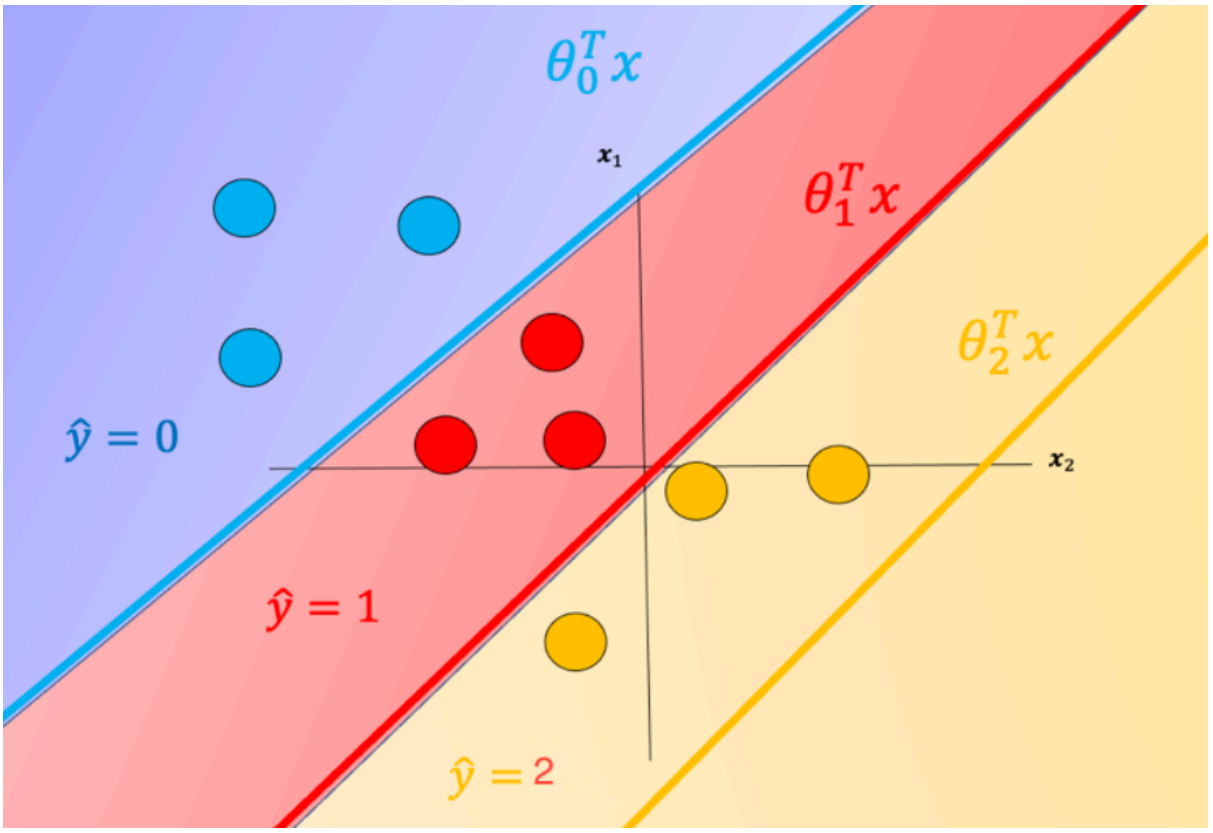
The training procedure is almost identical to **Logistic Regression** using cross-entropy, but the prediction is different. Consider the three-class example where **y** belongs to **{0,1,2}** i.e. **y** can equal 0,1,2. We would like to classify ***x***. We can use the **SoftMax** function to generate a probability of how likely the sample belongs to each class. We then can make a prediction using the ***argmax*** function:

$$\hat{y} = argmax_i(softmax(x, i)) \quad (2)$$

Geometric Interpretation

Each $\boldsymbol{\Theta_i^T x}$ is the equation of a hyperplane, we plot the intersection of the three hyperplanes with 0 in Fig 1 as coloured lines, in addition, we

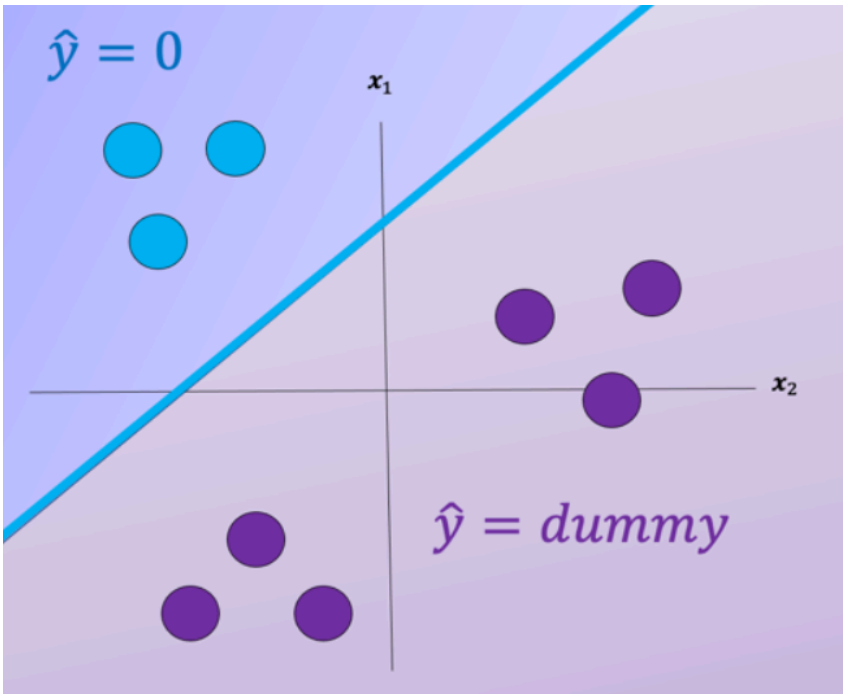
can overlay several training samples. We also shade the regions where the value of $\Theta_i^T \boldsymbol{x}$ is largest, this also corresponds to the largest probability. This colour corresponds to where a sample $\boldsymbol{x}$ would be classified. For example, if the input is in the blue region, the sample would be classified as $\hat{y}=0$, if the input is in the red region it would be classified as $\hat{y}=1$, and the yellow region $\hat{y}=2$.



One vs. All (One-vs-Rest)

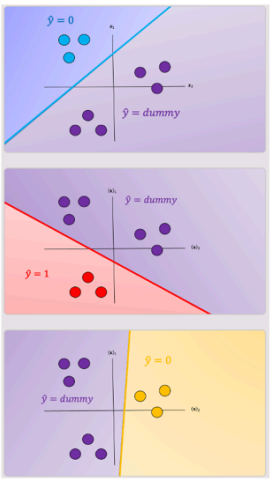
For one-vs-all classification, if we have K classes, we use K two-class classifier models, the number of class labels present in the dataset is equal to the number of generated classifiers. First, we create an artificial class we will call this "dummy" class. For each classifier, we split the data into two classes. We take the class samples we would like to classify; the rest of the samples will be labelled as a dummy class. We repeat the process for each class. To make a classification, we can use the majority vote or the classifier with the highest probability, disregarding the probabilities generated for the dummy class.

For each class, we take the class samples we would like to classify, and the rest will be labelled as a “dummy” class. For example, to build a classifier for the blue class we simply assign all other labels that are not in the blue class to the Dummy class, we then train the classifier accordingly. The result is shown in Fig 3 where the classifier predicts blue **ŷ=0** and in the purple region where we have our “dummy class” **ŷ = dummy**.



We repeat the process for each class, the actual class is shown with the same colour and the corresponding dummy class is shown in purple. The colour of the space is the actual classifier prediction shown in the same manner as above.

There are several ways to reduce this ambiguous region, you can use the output based on the output of the linear function this is called the fusion rule. We can also use the probability of a sample belonging to the actual class as shown in Fig 6, where we select the class with the largest probability in this case **ŷ=0**; we disregard the dummy values. These probabilities are scores, as the probabilities are between the dummy class and the actual class not between classes. Just a note packages like Scikit-learn can output probabilities for SVM.



$P(\hat{y}_0 = 0|\mathbf{x}) = 0.99$

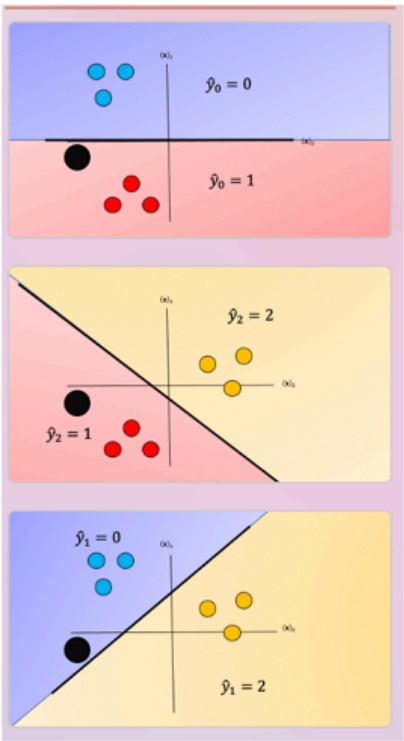
$P(\hat{y}_1 = 1|\mathbf{x}) = 0.89$

$P(\hat{y}_2 = 1|\mathbf{x}) = 0.79$

One-vs-One Classification

In One-vs-One classification, we split up the data into each class; we then train a two-class classifier on each pair of classes. For example, if we have classes 0,1 and 2, we would train one classifier on the samples that are class 0 and class 1, a second classifier on samples that are of class 0 and class 2, and a final classifier on samples of class 1 and class 2. Fig 7 is an example of class 0 vs class 1, where we drop training samples of class 2. Using the same convention as above where the color of the training samples is based on their class. The separating plane of the classifier is in black. The colour represents the output of the classifier for that particular point in space.

We repeat the process for each pair of classes, in Fig 8. For K classes, we have to train $K(K-1)/2$ classifiers. So if $K=3$, we have $(3\times2)/2=3$ classes.



$\hat{y}_0$	$\hat{y}_1$	$\hat{y}_2$
1	1	0

Implementation

https://labs.cognitiveclass.ai/v2/tools/jupyterlab?ulid=ulid-ba0d6c176d863df511f560c7b2e1fdd3bfcc7ec6

Go through the GitHub Repo…

Clustering

Clustering for segmentation

It is partitioning a customer base into groups of individuals with similar characteristics. It allows businesses to target specific groups of customers, and allocate marketing resources. Customers can be grouped based on several factors, including age, gender, interest etc. This can be done using the **Clustering** on **Unsupervised Model**, after that we can name the clusters according to the factors.

What is Clustering?

It means finding clusters in a dataset, unsupervised.

What is a Cluster?

It is a group of data points or objects in the cluster, and dissimilar to data points in other clusters.

Clustering vs Classification

Classification algorithms predict categorical classed labels. This means assigning instances to predefined classes such as defaulted or not defaulted. **Classification** is a **Supervised Learning** where each training data instance belongs to a particular class.

In **Clustering**, the data is unlabeled and the process is **Unsupervised**.

Clustering Applications:

- Retail Marketing
    - Identifying buying patterns of customers
    - Recommendation
- Banking
    - Fraud Detection
- Insurance
    - Insurance risk of customers
- Publication

- Medicines
    - Patient behaviours
- Biology
    - Genetic markings

Why Clustering?
- Exploratory data analysis
- Summary Generation
- Outlier detection
- Finding duplicates
- Pre-processing step

Clustering Algorithms:
- **Partitioned-based Clustering**: It is relatively efficient. Example: k-Means, k-Median, Fuzzy c-Means
- **Hierarchical Clustering**: Produces Trees of clusters. Example: Agglomerative, Divisive
- **Density-based Clustering**: Produces arbitrarily shaped clusters. Example: DBSCAN

k-Means Clustering

This algorithm can be used for Customer Segmentation. It can group data only Unsupervised based on the similarity of customers to each other.

It is a **Partitioning Clustering**. It divides the data into **k** non-overlapping subsets (clusters) without any cluster-internal structure. Examples within a cluster are very similar and across different clusters are very different.

Determine the similarity or dissimilarity

k-Means tries to minimize the intra-cluster distances and maximize the inter-cluster distances.

1 - dimensional similarity/distance

We can use the basic distance formula to calculate the distance between 2 data points.

Minkowski Distance

$$\text{Dis}\ (x_1, x_2) = \sqrt{\sum_{i=0}^{n} (x_{1i} - x_{2i})^2}$$

Multi-dimensional similarity/distance

We can also use this formula for more than 1 feature as well.

How does k-Means Clustering work?

For simplicity, our dataset has only 2 features: the age and income of customers. This means it is a 2D space. We show the distribution of customers using a scatter plot. At first, we need to determine the number of clusters.

The key concept of the **k-Means** algorithm is that it randomly picks centre points for each cluster. It means we need to initialize **k** which represents the number of clusters. There 2 approaches to choose the centroid:

1. We can randomly choose **k** observations from the dataset and use them as initial means.
2. We can create **k** random points as centroids of the clusters which is our choice.

After the initialization step, we have to assign each customer to the closest centre. For this, we need to calculate the distance of each point from the centres. This will form a **Distance Matrix** which stores the distance of the data point from each centre. The main objective of **k-means Clustering** is to minimize the distance of data points from the centroid of the cluster and maximize the distance from other cluster centroids.

The error is the total distance of each point from its centroid.

SSE = the sum of the squared differences between each point and its centroid.

$$SSE = \sum_{1}^{n} (x_i - C_j)^2$$

Sometimes, the clusters are not formed better. To do so, each centroid moves according to its members. The centroid of each of the 3 clusters becomes the new mean. Now, again calculate the distance.

K-means is an iterative algorithm and we have to repeat steps 2-4 until the algorithm converges. In each iteration, it will move the centroid, calculate the distance from new centroids and assign data points to the nearest centroids. It results in clusters with minimum errors or the most dense clusters.

k-Means clustering algorithm

- Randomly placing **k** centroids, one for each cluster
- Calculate the distance of each point from each centroid
- Assign each data point (object) to its closest centroid, creating a cluster
- Recalculate the position of the **k** centroids
- Repeat the steps until the centroids don't move

k-Means accuracy

- External approach
    - compare the clusters with ground truth, if it is available
- Internal approach
    - average the distance between data points within a cluster

Choosing k

There are some approaches to address the problem, but one of the techniques that are commonly used is to run the clustering across the different values of **k** and look at a metric of accuracy for

clustering. This metric can be the mean, distance between data points and their cluster's centroid, which indicates how dense our clusters are. Then looking at the change of the metric, can find the best value of **k**.
We can also use the **Elbow Method**, we take the value of **k** where the **Elbow Point** occurs in the plotted graph.

Important Note!
GitHub Repository Link: Machine Learning Repository